



UFR Sciences et Technologies
Département Informatique
M2 Génie Logiciel

* * *

2023-2024

BIG DATA

Marie NDIAYE

NoSQL

Que reproche-t-on aux bases de données relationnelles ?

- Lorsque l'on traite de très gros volume de données
 - **Augmentation des ressources coûteuse** : Augmenter le nombre de CPU (Processeurs), le nombre de RAM et de disques durs → **Scaling-Up** (Développement Verticale)
 - **Latence** (Mécanismes pour respecter les propriétés ACID : verrous, journalisation, etc.) → Perte de performance

Attention !

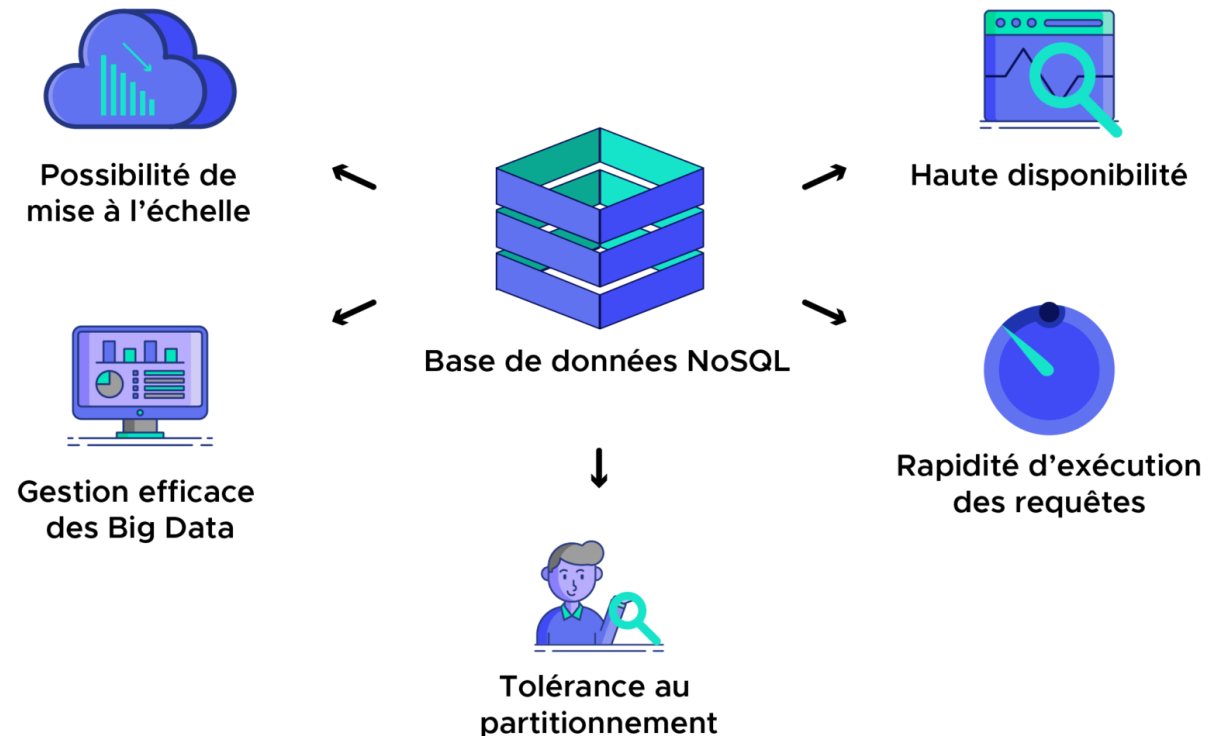
Avec une application et un design relationnel bien construit, les limites des systèmes relationnels ne sont pas facilement atteintes. Il faut compter plusieurs centaines de requêtes par seconde, avec des ensembles de données contenant soit beaucoup de petites entrées, soit des entrées de taille importante avant d'arriver aux limites d'une base de données relationnelle.

Bases de données relationnelles : Propriétés ACID

- ACID (atomicité, cohérence, isolation et durabilité) : ensemble de propriétés qui garantissent que les transactions préservent la cohérence de la *BD*, c'est à dire qu'elle transforme la *BD* d'un état cohérent à un autre
 - **Atomicité** – Les transactions constituent l'unité logique de travail, toute la transaction est exécutée ou bien rien du tout, mais jamais une partie seulement de la transaction.
 - **Cohérence** – Les transactions préservent la cohérence de la *BD*, c'est à dire qu'elle transforme la *BD* d'un état cohérent à un autre (sans nécessairement que les états intermédiaires internes de la *BD* au cours de l'exécution de la transaction respectent cette cohérence)
 - **Isolation** – Les transactions sont isolées les unes des autres, c'est à dire que leur exécution est indépendante des autres transactions en cours. Elles accèdent donc à la *BD* comme si elles étaient seules à s'exécuter, avec comme corollaire que les résultats intermédiaires d'une transaction ne sont jamais accessibles aux autres transactions.
 - **Durabilité** – Les transactions assurent que les modifications qu'elles induisent perdurent, même en cas de défaillance du système.

Qu'est-ce qu'une base de données NoSQL

- Une base de données NoSQL est une base de données « Not Only SQL » ou « pas uniquement SQL ».
 - Il est possible d'y stocker des données sous une forme non structurée, sans suivre de schéma fixe ;
 - Les jointures ne sont plus nécessaires ;
 - Le passage à l'échelle (scaling) est facilité.



<https://datascientest.com/nosql>

Historique : Une affaire de GAFAM

- **1998** → Le terme et le concept NoSQL furent inventés **par Carl Stroz** pour de désigner sa base de données relationnelle légère et open source.
- **2000** → Le **SGBD** basé sur les graphes, **Neo4j**, fut lancée par la société suédo-américaine Neo technology.
- **2004** → Google Bigtable,
- **2005** → Apache CouchDB.
- **2007** → Amazon Dynamo.
- **2008** → Facebook rend open source la base de données non-relationnelle Cassandra qu'elle utilisait en interne.

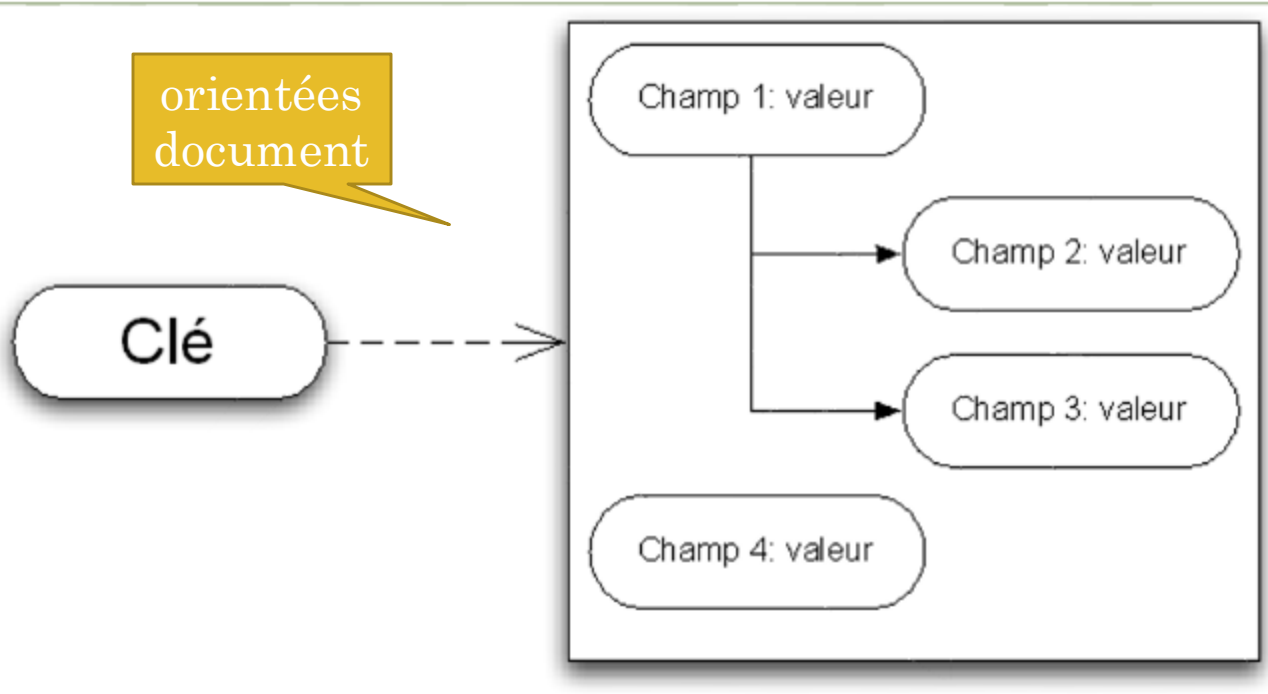
Les types de BD NoSQL

- Quatre grandes familles de base de données

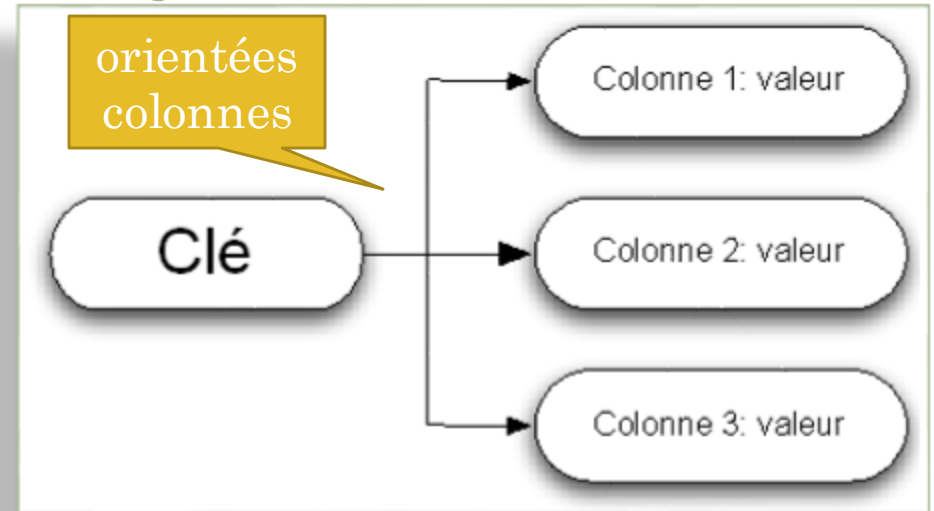
Clé-valeur



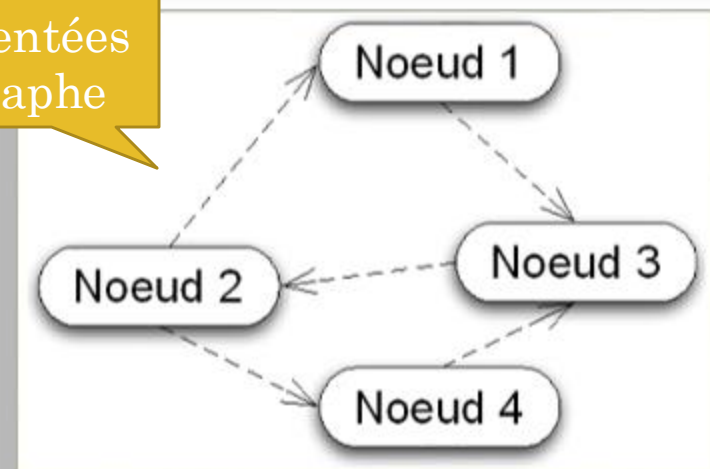
orientées document



orientées colonnes



orientées graphe



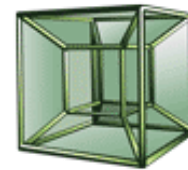
Exemples

- <https://www.lebigdata.fr/nosql-essentiel-a-savoir>

APACHE
HBASE

 **Cassandra**


CouchDB
relax



riak



mongoDB

HYPERTABLE INC



Neo4j



redis

Avantage / Inconvénients

- **Avantages**

- **Distribution facile** : données stockées dans des serveurs différents en local ou en cloud ; scalabilité horizontale (Scaling-Out) possible.
- **Disponibilité de la donnée** : La réplication des données (découlant de la distribution) permet de réduire la latence et la charge de travail des data centers.
- **Flexibilité**
 - S'adaptent à tous les types de données ; prise en charge des données structurées ou non structurées de la même manière.
 - Compatibilité avec les principaux langages de programmation.
 - Le schéma des données peut être modifié facilement sans interruption.

- **Inconvénients**

- **Absence de règles de standardisation** : les données sont dé-normalisées et traitent avec des enregistrements → duplications de données et autres anomalies.
- **Capacités de requêtes limitées** : jointures complexes non supportées.
- Difficultés à maintenir des **valeurs de clé uniques** lorsque le volume de données augmente.
- **Propriétés ACID** non prises en charge (à programmer).

HBase



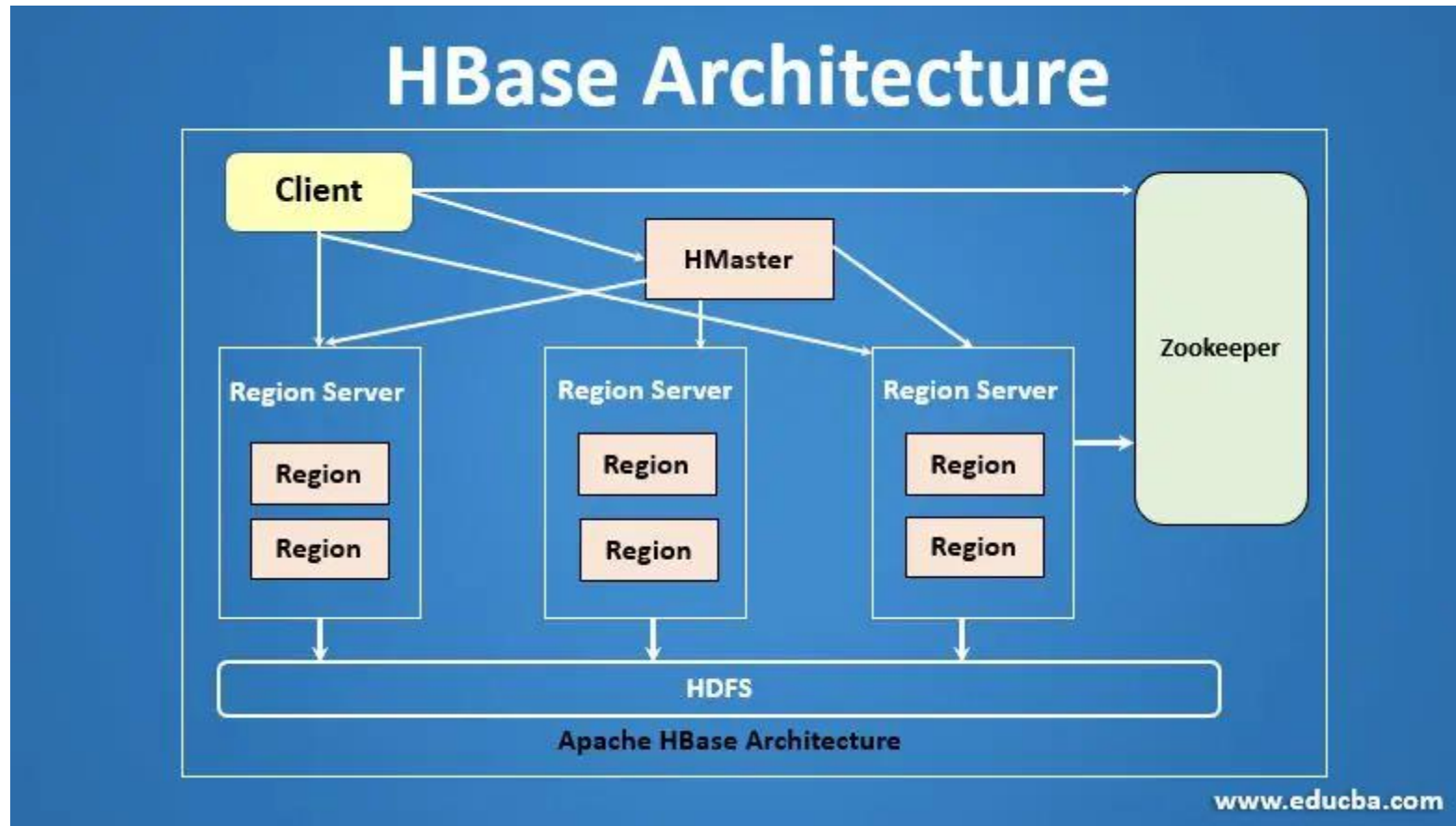
Présentation

- HBase est un SGBD non-relationnelles (NoSQL) orientée colonnes, distribué et évolutif horizontalement qui fonctionne au-dessus de HDFS. Il fournit un accès en lecture/écriture en temps réel aux données Hadoop.
- Il a été lancé comme projet par la société américaine Powerset à des fins de recherche en langage naturel. Il est devenu un projet Apache de haut niveau depuis 2010.
- Il est écrit en Java et il est open-source.
- Il s'agit d'un modèle de données similaire à la Big Table de Google, connue à l'origine sous le nom de Google BigTable au départ.
- HBase peut héberger de très grandes tables telles que des milliards de lignes et des millions de colonnes. HBase a la capacité de stocker une quantité massive de données de terabytes à des pétaoctets.
- Il peut combiner des sources de données qui utilisent une grande variété de structures et de schémas différents.

Les composants

- L'architecture de HBase a 3 principaux composants :
 - **Hmaster** – L'implémentation du serveur maître dans HBase est HMaster. C'est un processus dans lequel les régions (ensembles de lignes de tables HBase) sont assignées au serveur de région (Region Server) ainsi que les opérations DDL (créer, supprimer la table). Il surveille toutes les instances de serveur de région présentes dans le cluster. HMaster possède de nombreuses fonctionnalités comme le contrôle de l'équilibrage de la charge, le basculement, etc.
 - **Region Server** - Les tables HBase sont divisées horizontalement par plage de clés de ligne en régions. Les régions sont les éléments de base du cluster HBase qui consistent en la distribution des tables et sont composées de familles de colonnes. Region Server fonctionne sur HDFS DataNode qui est présent dans le cluster Hadoop. Les régions du serveur de régions sont responsables de plusieurs choses, comme la manipulation, la gestion, l'exécution ainsi que la lecture et l'écriture des opérations HBase sur cet ensemble de régions. La taille par défaut d'une région est de 256 Mo.
 - **Zookeeper** – Il est comme un coordinateur dans HBase. C'est un serveur esclave qui est responsable du service et de la gestion des serveurs de régions. Il fournit des services tels que le maintien des informations de configuration, l'attribution de noms, la synchronisation distribuée, la notification de défaillance du serveur, etc. Les clients communiquent avec les serveurs régionaux via zookeeper.

L'architecture

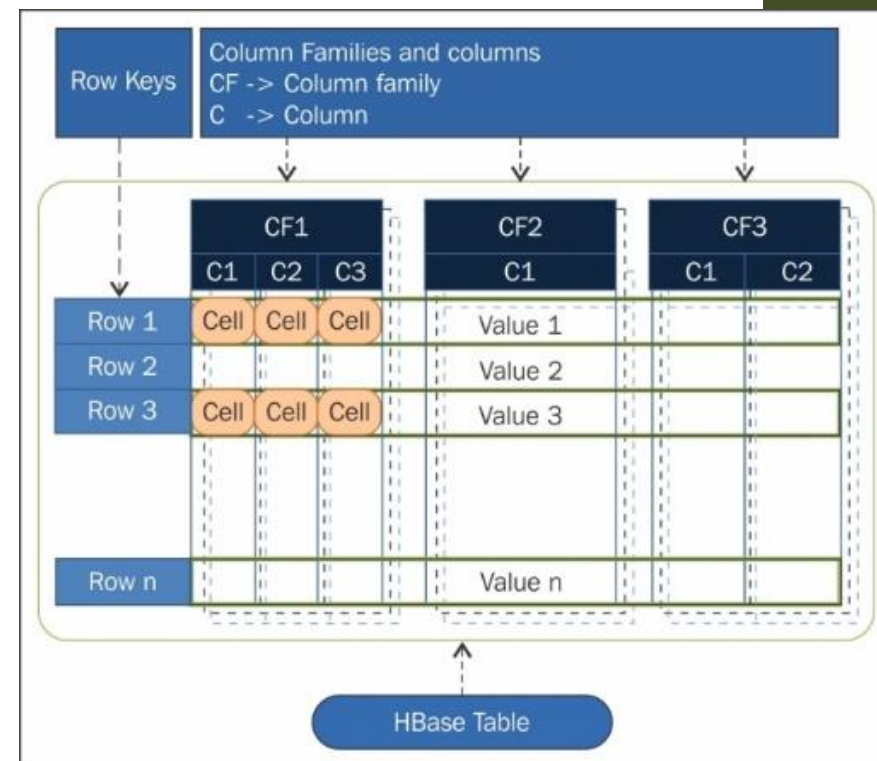


<https://www.educba.com/hbase-architecture/>

Le modèle de données

Row Key	Column Family			
Row Key	Customers		Products	
Customer ID	Customer Name	City & Country	Product Name	Price
1	Sam Smith	California, US	Mike	\$500
2	Arijit Singh	Goa, India	Speakers	\$1000
3	Ellie Goulding	London, UK	Headphones	\$800
4	Wiz Khalifa	North Dakota, US	Guitar	\$2500

Figure: HBase Table



<https://insatunisia.github.io/TP-BigData/tp4/>

Les HFiles

- Les données dans HBase sont stockées sous forme de *HFiles*, par colonnes, dans HDFS. Chaque *HFile* se charge de stocker des données correspondantes à une *column family* particulière.

- **Table** ----->
 - Contains column-families
- **Column family** ----->
 - Logical and physical grouping of columns
- **Column** ----->
 - Exists only when inserted
 - Can have multiple versions
 - Each row can have different set of columns
 - Each column identified by it's key
- **Row key** ----->
 - Implicit primary key
 - Used for storing ordered rows
 - Efficient queries using row key

HBTABLE	
Row key	Value
11111	cf_data: { 'cq_name' : 'name1', 'cq_val' : 1111} cf_info: { 'cq_desc' : 'desc11111'}
22222	cf_data: { 'cq_name' : 'name2', 'cq_val' : 2013 @ ts = 2013, 'cq_val' : 2012 @ ts = 2012 }

HFile

```
11111 cf_data cq_name name1 @ ts1
11111 cf_data cq_val 1111 @ ts1
22222 cf_data cq_name name2 @ ts1
22222 cf_data cq_val 2013 @ ts1
22222 cf_data cq_val 2012 @ ts2
```

HFile

```
11111 cf_info cq_desc desc11111 @ ts1
```

Références

- NoSQL
 - <https://stph.scenari-community.org/bdd/0/co/traUC004.html>
 - <https://www.data-bird.co/sql/nosql>
 - <https://datascientest.com/nosql>
 - <https://www.lebigdata.fr/nosql-essentiel-a-savoir>
- Hbase
 - <https://www.editions-eni.fr/video/hbase-introduction-a-la-base-de-donnees-nosql-d-hadoop-vthba>
 - <https://www.geeksforgeeks.org/architecture-of-hbase/>